

**Shutdown the StructuredTaskScope:** Ensure that the StructuredTaskScope is shut down at the end of the try block.

```
import java.util.concurrent.ExecutionException;
import java.util.concurrent.Future;
import java.util.concurrent.StructuredTaskScope;
import java.util.concurrent.StructuredTaskScope.Subtask;
public class StructuredConcurrencyExample {
    public static void main(String[] args) throws
        InterruptedException, ExecutionException, Exception {
        try (var scope = new StructuredTaskScope<Object>()) {
            Callable<String> task1 = () -> {
                Thread.sleep(1000);
                return "Task A completed";
            };
            Callable<String> task2 = () -> {
                Thread.sleep(500);
                return "Task B completed";
            };
            Subtask<String> subtask1 = scope.fork(task1);
            Subtask<String> subtask2 = scope.fork(task2);
            scope.join();
            System.out.println(subtask1.get());
            System.out.println(subtask2.get());
        }
    }
}
```

## Quantum-resistant module for improved security

Java 24 includes a quantum-resistant module designed to improve the security of Java applications. It provides an implementation of the quantum-resistant Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM). This feature is a step towards post-quantum readiness and aims to deliver post-quantum cryptographic resilience.

Module-Lattice Digital Signature Algorithm (ML-DSA) implementations will be provided for the KeyPairGenerator API to generate ML-DSA key pairs, the Signature API to sign and verify ML-DSA signatures, and the KeyFactory API to convert ML-DSA keys to and from their encodings.

A new standard algorithm family name, 'ML-DSA', will be defined in the Java Security Standard Algorithm Names Specification for the KeyPairGenerator, Signature, and KeyFactory APIs.

ML-KEM is for key establishment, while ML-DSA is for digital signatures.

Generating ML-DSA key pairs can be done in one of three ways:

**Choice 1:** Create a KeyPairGenerator with the family name and initialise it using a parameter-set name:

```
KeyPairGenerator g = KeyPairGenerator.getInstance("ML-DSA");
g.initialize(NamedParameterSpec.ML_DSA_44);
```

```
KeyPair kp = g.generateKeyPair(); // an ML-DSA-44 key pair
```

**Choice 2:** If the KeyPairGenerator is not initialised with a specified parameter set, the implementation will default to ML-DSA-65:

```
KeyPairGenerator g = KeyPairGenerator.getInstance("ML-DSA");
KeyPair kp = g.generateKeyPair(); // an ML-DSA-65 key pair
```

**Choice 3:** Instantiate a KeyPairGenerator directly using a specified parameter-set name:

```
KeyPairGenerator g = KeyPairGenerator.getInstance("ML-DSA-87");
KeyPair kp = g.generateKeyPair(); // an ML-DSA-87 key pair:
```

```
ML-KEM:
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.NoSuchAlgorithmException;
public class MLKEMTest {
    public static void main(String[] args) {
        // TODO Auto-generated method stub
        try {
            KeyPairGenerator keyPairGenerator =
                KeyPairGenerator.getInstance("ML-KEM");
            KeyPair keyPair = keyPairGenerator.
                generateKeyPair();
            System.out.println("Public Key (ML-KEM): " +
                keyPair.getPublic());
            System.out.println("Private Key (ML-KEM): " +
                keyPair.getPrivate());
        } catch (NoSuchAlgorithmException e) {
            e.printStackTrace();
        }
    }
}
```

```
ML-DSA:
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.NoSuchAlgorithmException;
import java.security.Signature;
import java.security.InvalidKeyException;
import java.security.SignatureException;
import java.util.Base64;
public class MLDSATest {
    public static void main(String[] args) {
        try {
            KeyPairGenerator keyPairGenerator =
                KeyPairGenerator.getInstance("ML-DSA");
            KeyPair keyPair = keyPairGenerator.
                generateKeyPair();
```